

What is Spring Framework?

1. Introduction

Spring Framework is an open-source Java framework used to build **enterprise-level Java applications**.

It provides infrastructure and reusable features that make Java application development easier, especially for applications that require:

- Web development
- REST APIs
- Database integration
- Transaction management
- Security
- Dependency management
- Testing
- Integration with other frameworks and technologies

One of the most important ideas behind Spring is:

Instead of our application manually creating and managing objects, we can allow the Spring Container to create, configure, connect, and manage those objects.

This is mainly achieved through:

- **IoC — Inversion of Control**
 - **DI — Dependency Injection**
-

2. Why Was Spring Framework Introduced?

Before Spring became popular, enterprise Java development often depended heavily on technologies such as:

- Servlets
- JSP
- JDBC
- EJB
- JNDI
- XML configuration
- Application servers

Developers had to write significant amounts of infrastructure and boilerplate code.

For example, a developer working directly with JDBC might need to:

1. Load/configure the database driver.
2. Create a database connection.
3. Create a statement.
4. Execute the SQL query.
5. Process the `ResultSet`.
6. Handle exceptions.
7. Commit or roll back transactions.
8. Close database resources.

Spring simplified many of these common enterprise-development responsibilities.

3. Major Problems Spring Helps Solve

3.1 Tight Coupling

Consider the following Java code:

```
public class EmployeeService {  
  
    private EmployeeRepository repository = new EmployeeRepository();  
  
}
```

`EmployeeService` is directly creating its dependency.

This creates **tight coupling**.

If we later want to replace:

`EmployeeRepository`

with:

`JpaEmployeeRepository`

or:

```
MockEmployeeRepository
```

we need to modify `EmployeeService`.

Spring addresses this using **Dependency Injection**.

Example:

```
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }
}
```

Now `EmployeeService` doesn't create the repository.

Spring provides the required repository object.

This produces:

```
Loose Coupling
  ↓
Better Maintainability
  ↓
Better Testability
  ↓
Easier Changes
```

4. What Is Inversion of Control (IoC)?

Normally, a Java developer controls object creation.

Example:

```
EmployeeService service = new EmployeeService();
```

Here, the developer is responsible for creating the object.

With Spring:

```
@Service
public class EmployeeService {

}
```

Spring can create and manage the `EmployeeService` object.

The control of object creation is therefore moved from:

Developer/Application

to:

Spring Container

This concept is called:

Inversion of Control — IoC

Traditional Java

```
Application
  ↓
Creates Objects
  ↓
Manages Dependencies
  ↓
Uses Objects
```

Spring

```
Application
  ↓
Spring Container
  ↓
Creates Objects
  ↓
```

Injects Dependencies



Manages Objects

5. What Is Dependency Injection?

Dependency Injection (DI) is a technique in which the dependencies required by a class are provided from outside instead of the class creating them itself.

Suppose:

```
EmployeeController
    ↓
EmployeeService
    ↓
EmployeeRepository
    ↓
Database
```

The controller depends on the service.

The service depends on the repository.

Instead of manually creating these objects:

```
EmployeeRepository repository = new EmployeeRepository();
EmployeeService service = new EmployeeService(repository);
EmployeeController controller = new EmployeeController(service);
```

Spring can create and connect them automatically.

Example:

```
@RestController
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }
}
```

```
}  
}
```

```
@Service  
public class EmployeeService {  
  
    private final EmployeeRepository employeeRepository;  
  
    public EmployeeService(EmployeeRepository employeeRepository) {  
        this.employeeRepository = employeeRepository;  
    }  
}
```

Spring identifies these dependencies and injects the appropriate objects.

6. Types of Dependency Injection

Spring commonly supports three approaches.

Constructor Injection

```
@Service  
public class EmployeeService {  
  
    private final EmployeeRepository employeeRepository;  
  
    public EmployeeService(EmployeeRepository employeeRepository) {  
        this.employeeRepository = employeeRepository;  
    }  
}
```

Constructor injection is generally the **recommended approach**.

Advantages:

- Dependencies can be immutable.
 - Easy to test.
 - Required dependencies are clear.
 - Helps avoid partially initialized objects.
 - Works without `@Autowired` when there is only one constructor.
-

Setter Injection

```
@Service
public class EmployeeService {

    private EmployeeRepository employeeRepository;

    @Autowired
    public void setEmployeeRepository(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }
}
```

Setter injection can be useful for optional or changeable dependencies.

Field Injection

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

}
```

Although simple, field injection is generally discouraged for production code because it makes dependencies less explicit and makes plain unit testing more difficult.

7. What Is the Spring Container?

The **Spring Container** is one of the core components of Spring Framework.

It is responsible for:

- Creating objects
- Configuring objects
- Injecting dependencies
- Managing object lifecycle
- Maintaining bean information

- Providing objects when required

The objects managed by Spring are called:

Spring Beans

For example:

```
@Service
public class EmployeeService {
}
```

Spring can detect this class and create an object.

Conceptually:

```
EmployeeService object
    ↓
Managed by Spring
    ↓
Spring Bean
```

8. Spring Bean

A **Spring Bean** is simply an object whose lifecycle is managed by the Spring IoC Container.

For example:

```
@Component
public class EmailService {
}
```

Spring can create an `EmailService` object and maintain it inside its container.

Conceptually:

```
Spring Container
|
├─ EmployeeController Bean
```


- └─ EmployeeService Bean
- └─ EmployeeRepository Bean
- └─ EmailService Bean

9. How Can We Create Spring Beans?

There are several approaches.

@Component

```
@Component
public class MessageService {
}
```

Generic Spring-managed component.

@Service

```
@Service
public class EmployeeService {
}
```

Typically used for the **business/service layer**.

@Repository

```
@Repository
public class EmployeeRepository {
}
```

Typically used for the **persistence/data-access layer**.

@Controller

```
@Controller
public class EmployeeController {
}
```

Used for Spring MVC controllers, commonly when returning views.

@RestController

```
@RestController
public class EmployeeController {
}
```

Commonly used for REST APIs.

@Bean

We can also explicitly create a bean.

```
@Configuration
public class AppConfig {

    @Bean
    public MessageService messageService() {
        return new MessageService();
    }
}
```

10. Bean Lifecycle

A simplified Spring Bean lifecycle is:

```
Application Starts
    ↓
Spring Container Starts
    ↓
Bean Definition Identified
    ↓
Bean Object Created
    ↓
Dependencies Injected
    ↓
Initialization
    ↓
Bean Ready to Use
    ↓
```

Application Running



Bean Destruction

Spring provides different mechanisms to participate in bean initialization and destruction.

For example:

```
@PostConstruct
public void init() {
    System.out.println("Bean initialized");
}
```

```
@PreDestroy
public void destroy() {
    System.out.println("Bean destroyed");
}
```

11. Bean Scopes

Spring supports different bean scopes.

Common scopes include:

Scope	Meaning
Singleton	One bean instance per Spring container
Prototype	New bean instance whenever requested
Request	One instance per HTTP request
Session	One instance per HTTP session
Application	One instance for the web application's servlet context

The default scope is:

Singleton

Example:

```
@Service
@Scope("prototype")
public class EmployeeService {
}
```

12. Spring Container Types

Two important container APIs are:

BeanFactory

`BeanFactory` is the basic IoC container.

It provides fundamental functionality for:

- Bean creation
- Bean configuration
- Dependency injection

ApplicationContext

`ApplicationContext` is a more feature-rich container built on Spring's bean-factory capabilities.

It provides additional features such as:

- Event handling
- Internationalization
- Resource loading
- Integration with Spring enterprise features

In modern Spring applications, **ApplicationContext** is used much more commonly.

Conceptually:

```
BeanFactory
  ↓
Basic Container Features

ApplicationContext
  ↓
BeanFactory Features
+
```

```
Events
+
Resources
+
Internationalization
+
Enterprise Integration
```

13. Major Modules of Spring Framework

Spring Framework is not a single library. It consists of multiple modules.

Core Container

Includes fundamental functionality related to:

- IoC
- Dependency Injection
- Beans
- ApplicationContext
- Spring Expression Language

Conceptually:

```
Spring Core Container
|
├─ Core
├─ Beans
├─ Context
└─ Expression
```

14. Spring Web / Spring MVC

Spring provides support for building web applications.

Spring MVC follows the:

Model-View-Controller

architecture.

A traditional MVC flow can look like:

```
Browser
↓
Controller
↓
Service
↓
Repository
↓
Database
```

The controller processes the HTTP request.

The service contains business logic.

The repository communicates with the database.

15. Spring MVC Architecture

A simplified Spring MVC request flow is:

```
Client / Browser
↓
HTTP Request
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
↓
Database
↓
Repository
↓
```

```
Service
  ↓
Controller
  ↓
HTTP Response
  ↓
Client
```

`DispatcherServlet` is an important component in Spring MVC.

It works as a **Front Controller**, receiving requests and coordinating their processing through the Spring MVC infrastructure.

16. Spring JDBC

Spring provides JDBC support to simplify database operations.

Traditional JDBC can require substantial repetitive code around:

- Connections
- Statements
- Result sets
- Exception handling
- Resource cleanup

Spring JDBC reduces this boilerplate.

A common class is:

```
JdbcTemplate
```

Example:

```
@Repository
public class EmployeeRepository {

    private final JdbcTemplate jdbcTemplate;

    public EmployeeRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

```
}  
}
```

17. Spring ORM

Spring provides integration with ORM technologies.

ORM means:

Object Relational Mapping

Spring can integrate with technologies such as Hibernate and JPA.

This allows developers to work with Java objects representing database records.

For example:

```
@Entity  
public class Employee {  
  
    @Id  
    private Long id;  
  
    private String name;  
}
```

Conceptually:

```
Java Object  
    ↑  
ORM / JPA / Hibernate  
    ↓  
Database Table
```

18. Spring Transaction Management

Transactions are important when multiple database operations must succeed or fail together.

Example:

```
Transfer Money
  ↓
Deduct ₹1000 from Account A
  ↓
Add ₹1000 to Account B
```

If the first operation succeeds but the second fails, the transaction should normally be rolled back.

Spring supports declarative transaction management.

Example:

```
@Transactional
public void transferMoney() {

}
```

Conceptually:

```
Begin Transaction
  ↓
Operation 1
  ↓
Operation 2
  ↓
Success?
  ↙   ↘
Yes    No
  ↓     ↓
Commit Rollback
```

19. Spring AOP

AOP stands for:

Aspect-Oriented Programming

Some functionality is required across many parts of an application.

Examples:

- Logging
- Security
- Transaction management
- Auditing
- Performance monitoring

These are called **cross-cutting concerns**.

Without AOP:

```
Controller → Logging Code  
Service    → Logging Code  
Repository → Logging Code
```

With AOP, common behavior can be separated:

```
Logging Aspect  
    ↓  
Controller → Service → Repository
```

This improves separation of concerns.

20. Spring Security

Spring Security provides security infrastructure for Java applications.

It supports areas such as:

- Authentication
- Authorization
- Password encoding
- Role-based access control
- Method-level security
- Web application security

Example:

```
Client  
    ↓  
Login Request
```

```
↓  
Spring Security  
↓  
Authentication  
↓  
Authorization  
↓  
Application Resource
```

Spring Security is a separate project in the broader Spring ecosystem and integrates closely with Spring applications.

21. Spring Testing Support

Spring provides testing support for Spring-based applications.

Applications can be tested at different levels:

```
Unit Testing  
↓  
Integration Testing  
↓  
Web Layer Testing  
↓  
Database Testing
```

Spring's testing infrastructure integrates with modern testing frameworks such as JUnit.

22. Layered Architecture with Spring

A common Spring application can be organized into layers.

```
Client  
↓  
Controller Layer  
↓  
Service Layer  
↓  
Repository Layer
```

↓
Database

Controller Layer

Responsible for:

- Receiving HTTP requests
- Request validation/co-ordination
- Returning HTTP responses

Example:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {
}
```

Service Layer

Responsible for:

- Business logic
- Business validations
- Transaction coordination

Example:

```
@Service
public class EmployeeService {
}
```

Repository Layer

Responsible for:

- Database communication
- CRUD operations
- Persistence logic

Example:

```
@Repository
public class EmployeeRepository {
}
```

23. Complete Example

Consider an Employee Management application.

The client sends:

```
POST /employees
```

Request:

```
{
  "name": "Anand",
  "email": "anand@example.com"
}
```

Application flow:

```
Client
  ↓
EmployeeController
  ↓
EmployeeService
  ↓
EmployeeRepository
  ↓
Database
```

Controller:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService employeeService;
```

```

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }
}

```

Service:

```

@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }
}

```

Repository:

```

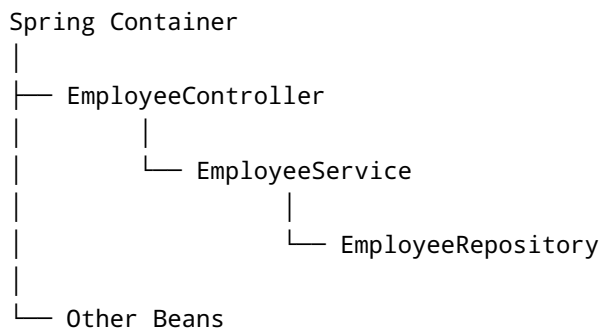
@Repository
public class EmployeeRepository {

}

```

Spring manages these objects and their dependencies.

Conceptually:



24. What Happens When a Spring Application Starts?

At a high level:

```
Start Application
  ↓
Create Spring Container
  ↓
Read Configuration
  ↓
Scan Components
  ↓
Find Bean Definitions
  ↓
Create Beans
  ↓
Resolve Dependencies
  ↓
Inject Dependencies
  ↓
Initialize Beans
  ↓
Application Ready
```

Spring can discover classes using component scanning.

For example:

```
@Component
@Service
@Repository
@Controller
@RestController
@Configuration
```

Spring uses this metadata to determine which objects need to be managed.

25. Traditional Java vs Spring

Traditional Java

```
EmployeeRepository repository =  
    new EmployeeRepository();  
  
EmployeeService service =  
    new EmployeeService(repository);
```

The developer manages object creation and wiring.

Spring

```
@Service  
public class EmployeeService {  
  
    private final EmployeeRepository repository;  
  
    public EmployeeService(EmployeeRepository repository) {  
        this.repository = repository;  
    }  
}
```

Spring manages the required dependency.

Therefore:

```
Traditional Java  
  
Developer  
↓  
Creates Objects  
↓  
Connects Objects  
↓  
Manages Dependencies
```

becomes:

```
Spring
```



```
Developer
  ↓
Defines Components
  ↓
Spring Container
  ↓
Creates Beans
  ↓
Injects Dependencies
  ↓
Manages Lifecycle
```

26. Advantages of Spring Framework

1. Loose Coupling

Dependencies can be injected instead of manually created.

2. Better Testability

Dependencies can be replaced with mocks or test implementations.

3. Modular Architecture

Developers can use the Spring modules they need.

4. Database Integration

Provides integration with JDBC, JPA, Hibernate, and transaction management.

5. Web Development

Spring MVC provides powerful web-development infrastructure.

6. Transaction Management

Transactions can be handled declaratively.

7. AOP Support

Cross-cutting concerns can be separated from core business logic.

8. Integration

Spring integrates with many Java and enterprise technologies.

9. Maintainability

Layered architecture and DI help make large applications easier to maintain.

10. Large Ecosystem

Spring Framework forms the foundation for a broad ecosystem including Spring Boot, Spring Data, Spring Security, Spring Cloud, and others.

27. Disadvantages / Challenges of Spring Framework

Spring is powerful, but there are challenges.

Configuration Complexity

Traditional Spring applications could require substantial configuration, particularly XML configuration.

Learning Curve

Beginners need to understand concepts such as:

- IoC
- DI
- Beans
- Bean lifecycle
- AOP
- MVC
- Transactions

Dependency Management

Developers historically had to manage many compatible dependencies manually.

Deployment Configuration

Traditional Spring web applications often required configuring and deploying applications to external servlet containers.

These challenges are among the reasons **Spring Boot** became popular.

28. Spring Framework vs Spring Boot

Spring Boot is built on top of the Spring ecosystem and simplifies the creation and configuration of Spring applications.

Spring Framework	Spring Boot
Core framework	Opinionated application-development approach built on Spring
More configuration may be required	Strong auto-configuration support
Dependencies may require more manual coordination	Starter dependencies simplify dependency setup
External server deployment was common traditionally	Embedded servers are commonly used
More setup/boilerplate	Reduced setup and boilerplate
Provides IoC, DI, MVC, AOP, etc.	Uses Spring features while simplifying application setup

Conceptually:

```
Spring Framework
↓
Powerful Enterprise Features
↓
But application setup can require configuration

+

Spring Boot
↓
Auto Configuration
+
Starter Dependencies
+
Embedded Server
+
Production-oriented Features
↓
Faster Application Development
```

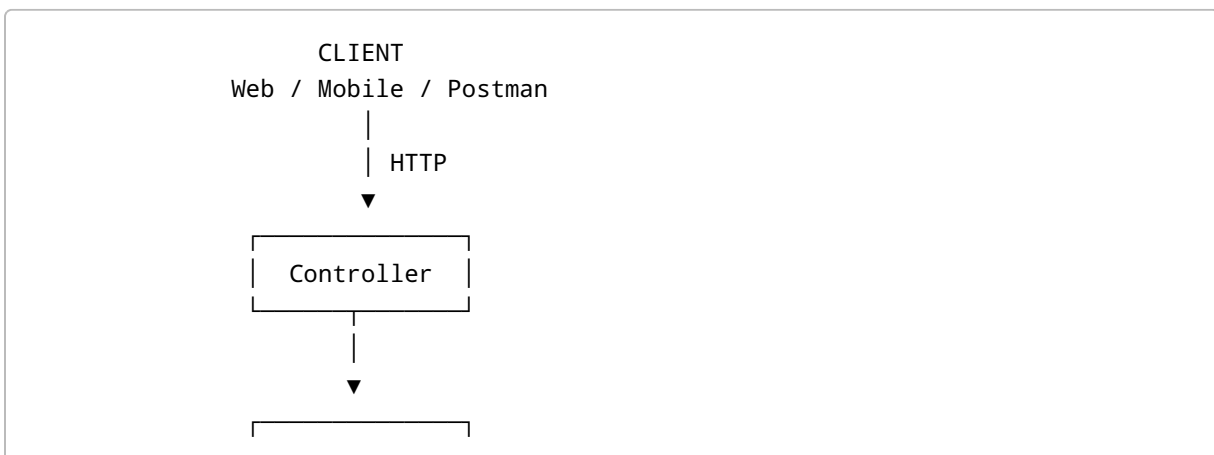
29. Important Spring Framework Annotations

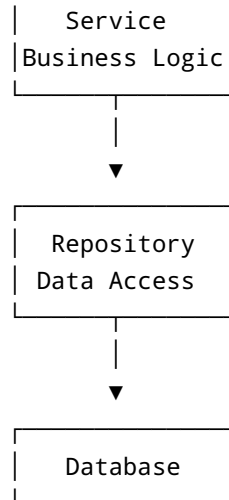
Some commonly encountered annotations include:

Annotation	Purpose
<code>@Component</code>	Generic Spring component
<code>@Service</code>	Service/business-layer component
<code>@Repository</code>	Persistence-layer component
<code>@Controller</code>	MVC controller
<code>@RestController</code>	REST controller
<code>@Autowired</code>	Requests dependency injection
<code>@Qualifier</code>	Selects a specific candidate
<code>@Primary</code>	Marks the preferred candidate
<code>@Configuration</code>	Configuration class
<code>@Bean</code>	Explicitly defines a bean
<code>@Scope</code>	Defines bean scope
<code>@Transactional</code>	Transaction management
<code>@Value</code>	Injects configuration values

30. Simple Real-Time Architecture

A typical Spring/Spring Boot backend application may look like:





Spring manages the components participating in this architecture.

31. Key Concepts Students Should Remember

The most important concepts are:

```
Spring Framework
|
|— IoC
|
|— Dependency Injection
|
|— Spring Container
|
|— Spring Beans
|
|— Bean Lifecycle
|
|— Bean Scopes
|
|— Spring MVC
|
|— JDBC / ORM Integration
|
|— Transaction Management
|
|— AOP
```

32. Interview Definition

A concise interview answer:

Spring Framework is an open-source Java framework for building enterprise applications. Its core features are Inversion of Control and Dependency Injection, where the Spring IoC Container creates, configures, wires, and manages application objects called beans. Spring also provides infrastructure for web development, data access, transactions, AOP, testing, and integration with enterprise technologies.

33. Simple Student Definition

For beginners:

Spring Framework is a Java framework that helps developers build applications by managing Java objects and their dependencies and by providing reusable infrastructure for common requirements such as web development, database access, transactions, and testing.

The easiest concept to remember is:

Without Spring

Developer creates objects
+
Developer connects objects
+
Developer manages dependencies

With Spring

Spring creates objects
+
Spring connects objects
+
Spring manages dependencies

This is the foundation for understanding **IoC, Dependency Injection, Spring Beans, Spring Container, and eventually Spring Boot.**